
Weave — Cognitive Runtime

Spécification technique · Operating System for Organizational Memory

Guillaume Flambard

Juillet 2026

Résumé exécutif

- Le déclencheur

Principes de conception

Vue d'architecture

Modèle de données

La mémoire vivante : le pipeline

- Consolidation & déduplication

Moteur d'émergence de Skills

Émergence & orchestration d'agents

Le Knowledge Graph

Recherche hybride avec provenance

Gateway LLM & Embeddings (multi-provider)

Surface API : REST · SSE · MCP

Architecture Rust (crates, ports & adapters)

Choix technologiques justifiés

Sécurité, permissions, isolation des mémoires

Observabilité

Déploiement

Roadmap

Comparaison

Vision long terme : un OS pour la mémoire organisationnelle

Annexe — Parcours de démonstration (~90 s)

Résumé exécutif

Weave est un *Cognitive Runtime* : une couche de mémoire organisationnelle partagée qui s'intercale **au-dessus des LLM** et **en-dessous des outils métier** (Slack, GitHub, Notion, Linear, Gmail...). Elle transforme en continu l'activité d'une équipe — conversations, décisions, PRs, documents — en **connaissance structurée, versionnée et interrogeable**, puis **fait émerger automatiquement des compétences (Skills) et des agents** à partir des schémas récurrents du travail réel.

Le problème que Weave résout n'est pas « créer des agents » — les plateformes savent le faire. Le problème est de leur donner une **mémoire collective exploitable** : un contexte partagé, durable, qui capitalise le travail de chacun et le restitue à tous, sans que personne n'ait à documenter quoi que ce soit.

*Ce document décrit un système **déjà implémenté** (MVP fonctionnel en Rust + Postgres + Next.js), pas une proposition. Chaque mécanisme décrit — émergence de skills, émergence d'agents, orchestration bornée, recherche hybride avec provenance, endpoint MCP — tourne et est vérifié de bout en bout.*

Le déclencheur

Lors d'un webinar Notion × PennyLane, l'intervenant PennyLane a décrit une capacité manquante : des **agents pré-établis partageant un contexte commun**, qui **généreraient eux-mêmes des skills** à partir du travail partagé de chacun, maintenant un contexte d'équipe consolidé. Weave est exactement ce système.

Principes de conception

1. **Event-sourcing.** Toute activité est un flux d'événements immuables. La mémoire est une projection restructurable, jamais une source de vérité opaque.
2. **Faits, pas transcriptions.** On ne stocke pas les conversations : on en distille des **faits atomiques** (décisions, questions, réponses).
3. **Mémoire indépendante de l'agent.** La connaissance vit dans le runtime, pas dans un agent. Tout agent — présent ou futur — en hérite instantanément.
4. **Émergence, pas programmation.** Skills et agents **naissent** des schémas récurrents ; personne ne les code.
5. **Provider-agnostic.** Le LLM et l'embedder sont des ports interchangeables (Ollama local, Claude, OpenAI, Mistral...).
6. **Ports & adapters.** La logique métier n'importe jamais l'infrastructure. Tout composant de stockage est remplaçable sans réécriture.
7. **Provenance native.** Chaque réponse expose **d'où** vient chaque affirmation (couche mémoire, auteur, source).
8. **Gouvernance.** Un agent émergent n'est activé qu'après validation humaine.

Vue d'architecture



Le principe cardinal du MVP : **une seule brique d'infrastructure (Postgres + pgvector)** couvre les quatre besoins habituellement éclatés — vecteurs (pgvector), recherche plein-texte / BM25 (tsvector), graphe de connaissances (arêtes en lignes + CTE récursives), et bus d'événements (canal in-process + table outbox). Comme la logique métier passe par des *traits* (ports), chaque brique se remplace ultérieurement par un adaptateur dédié **sans toucher au cœur**. Le MVP est donc un **sous-ensemble strict** de la cible entreprise : rien n'est jetable.

Modèle de données

Sept entités de premier plan, toutes portées par le crate `weave-core` (types purs, sans I/O).

Entité	Rôle	Champs clés
Event	observation immuable d'une source	<code>source</code> , <code>ts</code> , <code>actor</code> , <code>project</code> , <code>kind</code> , <code>payload</code> , <code>confidence</code> , <code>content_hash</code>
Fact	connaissance atomique distillée	<code>f_type(decision\ question\ answer\ fact)</code> , <code>author</code> , <code>topic</code> , <code>content</code> , <code>confidence</code> , <code>memory_level</code> , <code>embedding</code> , <code>superseded_by</code>
Entity	nœud du graphe	<code>name</code> , <code>kind(person\ component\ service\ concept)</code>
Relationship	arête du graphe	<code>src</code> , <code>dst</code> , <code>rel</code>
Pattern	signature récurrente	<code>signature</code> , <code>kind</code> , <code>occurrences</code> , <code>fact_ids[]</code>
Skill	compétence émergée	<code>name</code> , <code>trigger</code> , <code>body</code> , <code>sources[]</code> , <code>referents[]</code> , <code>derived_from_pattern</code> , <code>memory_level</code>
Agent	rôle qui agit	<code>role</code> , <code>domain</code> , <code>skills[]</code> , <code>scope</code> , <code>status(active\ pending)</code> , <code>derived_from</code>

Immutabilité & versioning. Les events sont immuables (dédup par `content_hash`). Les facts sont versionnés : une contradiction ne supprime rien, elle chaîne l'ancien fait via `superseded_by` — d'où un « time-travel » naturel.

Niveaux de mémoire. Chaque fact porte un `memory_level` : `personal` → `team` → `project` → `organization`. C'est l'axe de portée qui permet à un agent de piocher du plus intime au plus global, et d'afficher la provenance par couche.

La mémoire vivante : le pipeline

Chaque événement traverse le pipeline (crate `weave-pipeline`) :

```
Event → dédup → extraction (entités, faits, relations)
      → embeddings → liaison graphe → détection de contradiction (v1)
      → mise à jour des résumés → détection de pattern → matérialisation de skill
      → (cluster de skills) → émergence d'agent
```

Les étapes d'extraction sont exposées derrière le port `LlmGateway::extract` (un aller-retour LLM par événement en MVP ; découposables en étapes distinctes en v1). Le bus est un canal broadcast in-process qui alimente aussi le flux **SSE** temps réel du dashboard.

Consolidation & déduplication

- **Dédup d'événements** : hash stable sur les champs sémantiquement identifiants → rejouer deux fois le même flux n'ingère rien en double (idempotence).
- **Consolidation de faits** : les faits partageant une signature convergent vers un même *pattern* ; les réponses redondantes sont dédupliquées avant synthèse.
- **Contradictions (v1)** : à l'insertion d'un fait contredisant un fait existant (même sujet, polarité opposée), on chaîne `superseded_by` et on archive l'ancienne version plutôt que de l'écraser.
- **Résumés hiérarchiques (v1)** : à la manière de Git — résumé journalier → hebdomadaire → mensuel → vision stratégique — pour éviter l'explosion de tokens.

Moteur d'émergence de Skills

C'est le cœur différenciant. **Personne n'écrit les skills ; elles naissent du travail de l'équipe.**

Détection de pattern. Pour tout fait relevant d'un *thread suivi*, on calcule une **signature normalisée** (`normalize_signature`) : minuscule, suppression des mots-vides FR/EN, tokens saillants triés. Ainsi « How do I deploy to staging? » et « comment déployer staging » collapser sur la même signature. La signature s'ancre sur un **indice de thread stable de la source** (l'équivalent d'un `thread_ts` Slack ou d'un n° de PR), ce qui rend l'émergence **indépendante du provider LLM** : même si un modèle local reformule ou mal-étiquette un fait, le regroupement tient.

Matérialisation. Quand une signature dépasse un seuil d'occurrences (défaut 5) avec des réponses convergentes, le `MemoryUpdater` :

1. rassemble les réponses observées (recherche plein-texte sur la signature) ;
2. synthétise un **corps de runbook** via le `LlmGateway` ;
3. déduit les **référents** (auteurs des réponses) et les **sources** (`fact_ids`) ;
4. persiste un `Skill` et émet un événement `SkillEmerged` sur le flux live.

Exemple réel produit (extraction Ollama `qwen3.5:9b`) — la skill `bancaire-relancer-synchro`, née de 5 interactions, référent `nicolas` :

Relancer la synchronisation bancaire via Rails Runner et vérifier l'état sur Grafana ou les webhooks. 1. Exécutez `rails runner 'BankSync.rerun("client_id")'`. 2. Vérifiez le dashboard Bridge sur Grafana. 3. En cas d'accès limité, `BankSync.rerun(client_id)` en console et surveillez les webhooks Bridge.

Émergence & orchestration d'agents

Weave pousse la logique un cran plus loin : **les agents aussi émergent**.

Agent = objet mémoire de premier plan. Un rôle (system prompt) + un *scope* mémoire + les *skills* qu'il peut utiliser + un *domaine* de routage + un *statut*.

Émergence. Quand un **domaine** accumule assez de skills liées (seuil défaut 2), un **agent spécialiste** naît du cluster — par ex. `specialiste-finance-ops` regroupant `bancaire-relancer-synchro` et `rejouer-stripe-webhook`. Il naît en statut **pending**.

Gouvernance (human-in-the-loop). Un agent émergent **n'est jamais activé automatiquement**. Il reste en attente jusqu'à une **approbation humaine** explicite (endpoint `/agents/approve`, bouton « Approuver » dans l'UI). C'est la garantie de contrôle indispensable en contexte entreprise.

Orchestration bornée. Un agent généraliste prédéfini (`assistant-pennylane`) reçoit une tâche et exécute une boucle **plan** → **délègue** → **vérifie** :

1. **Retrieve** : contexte scopé (skills de l'agent + faits pertinents) ;
2. **Delegate** : si la tâche relève d'un domaine couvert par un spécialiste **actif**, il lui délègue (sous-agent) ;
3. **Verify** : un vérificateur valide le résultat ; sinon l'agent répond lui-même ;
4. **Trace** : chaque étape est tracée et affichée.

Garde-fous durs (non négociables) : profondeur de délégation max (2), nombre d'agents max par tâche (8), budget temps (180 s), et **validation humaine** des agents émergents. Ces plafonds bornent le coût et empêchent tout emballement récursif.

Trace réelle observée :

```
delegate  assistant-pennylane → specialiste-finance-ops (finance-ops)
answer    specialiste-finance-ops  · domaine finance-ops
verify    assistant-pennylane      · résultat accepté
```

Le Knowledge Graph

Les entités (personnes, composants, services, concepts) et leurs relations (`owns` , `depends_on` , `exposes` , `works_on` ...) forment un graphe de connaissances. En MVP il est stocké en Postgres (arêtes en lignes, traversée par CTE récursives) ; en v2 il migre vers **Kuzu** (base graphe embarquée, requêtes Cypher) derrière le même port `GraphStore` . Le graphe sert à : relier faits et skills à des entités, router l'émergence d'agents par domaine, et enrichir le contexte de réponse (voisinage d'une entité citée).

Recherche hybride avec provenance

Une requête agent combine trois signaux, fusionnés et dédoublés :

- **Vectoriel** : similarité cosinus via pgvector (`embedding <=> $query`).
- **Plein-texte / BM25** : `tsvector` + `ts_rank` .
- **Graphe** : voisinage des entités mentionnées (v1+).

Les faits récupérés sont **regroupés par couche mémoire** (`personal` → `team` → `project` → `organization`), et la réponse expose cette **provenance** : quelle couche, quel auteur, quelle skill utilisée. C'est ce qui rend une réponse **auditable** — critère décisif en environnement compta/finance.

Gateway LLM & Embeddings (multi-provider)

Deux ports :

- **LlmGateway** — `extract`, `synthesize_skill`, `answer`. Adaptateurs : **Ollama** (local, sans clé — défaut), **Claude** (Anthropic), et un extracteur **heuristique** hors-ligne. Sélection par `WEAVE_LLM_PROVIDER = ollama | claude | heuristic | auto`. Chaque adaptateur dégrade proprement vers l'heuristique en cas d'échec réseau : la démo ne peut pas planter.
- **EmbeddingGateway** — `embed`. Adaptateur MVP local (hachage n-gram normalisé, zéro dépendance) → Voyage / BGE / Jina en v1.

Vérifié en local sur `qwen3.5:9b` : extraction structurée, synthèse de skill, réponse générative avec provenance — le tout **sans aucune clé API**.

Surface API : REST · SSE · MCP

Méthode	Route	Rôle
POST	/replay	rejoue un flux (live)
POST	/reset	réinitialise un projet
GET	/events	flux SSE des événements pipeline
GET	/facts , /skills , /graph	lecture de la mémoire
POST	/ask	réponse agent + provenance des couches
GET	/agents	agents prédéfinis + émergents
POST	/agents/approve	active un agent émergent (gouvernance)
POST	/agents/run	orchestration + trace
POST	/mcp	endpoint MCP (ask_memory) pour agents externes

MCP (Model Context Protocol). Weave expose un endpoint JSON-RPC MCP avec un outil `ask_memory(project, question)` . **N'importe quel agent externe** (Claude, un IDE, un autre orchestrateur) se branche sur la mémoire partagée de l'équipe via un protocole standard — matérialisant exactement la vision « agents pré-établis piochant dans un contexte commun ».

Architecture Rust (crates, ports & adapters)

```
crates/  
  weave-core      types domaine purs (aucun I/O)  
  weave-store     ports (EventStore, FactStore, GraphStore, PatternStore,  
                  SkillStore, AgentStore) + adaptateur Postgres  
  weave-llm       LlmGateway + EmbeddingGateway (Ollama|Claude|heuristique)  
  weave-pipeline  bus + étapes → skills → agents + orchestrateur  
  weave-ingest    trait Connector (seam Slack/GitHub) + dataset seed  
  weave-api       Axum : REST + SSE + MCP  
  apps/web        dashboard Next.js (4 panneaux + agents + trace)
```

L'architecture **hexagonale** est la garantie de survie du MVP jusqu'à l'entreprise : `weave-core` ne connaît ni Postgres ni le réseau ; `weave-pipeline` ne parle qu'aux traits. Remplacer Postgres par Qdrant+Kuzu+NATS = écrire des adaptateurs, jamais réécrire la logique.

Choix technologiques justifiés

- **Rust + Axum + Tokio** : performance, sûreté mémoire, et un pipeline I/O-bound où l'async excelle. Un runtime de mémoire doit être fiable et frugal.
- **Postgres-as-everything (MVP)** : pgvector + tsvector + graphe relationnel + LISTEN/NOTIFY. Une dépendance à opérer au lieu de six → vitesse maximale sans compromettre le chemin d'évolution.
- **Ports & adapters** : seule façon honnête de garder le MVP fidèle à la cible.
- **Ollama par défaut** : tester une vraie IA en local, sans clé, sans coût.
- **MCP dès le jour 1** : interopérabilité standard avec l'écosystème d'agents.

Sécurité, permissions, isolation des mémoires

- **Isolation par projet (MVP)** : toute donnée est scopée `project` .
- **Scopes mémoire** : un agent déclare un `scope` (niveau minimal lisible) ; la récupération respecte la portée. Base d'un futur **RBAC** par couche (`personal` privé à l'utilisateur, `organization` global).
- **Multi-tenant (Enterprise)** : isolation par tenant (schémas Postgres ou RLS), chiffrement au repos, redaction PII à l'ingestion.
- **Gouvernance d'agents** : approbation humaine obligatoire avant activation ; journal d'audit des délégations (la trace d'orchestration).

Observabilité

Cible v1 : **OpenTelemetry** (traces distribuées sur le pipeline et l'orchestration), **Prometheus** (métriques : débit d'ingestion, taux d'émergence, latence LLM par provider, coût par tâche), **Grafana** (dashboards). Le budget-temps de l'orchestrateur et le compteur d'agents sont déjà des métriques de premier plan.

Déploiement

- **Local / démo** : `docker compose up` (Postgres+pgvector) + `cargo run` + `pnpm dev` . Ollama local pour l'IA.
- **v1** : image Docker unique pour l'API, Postgres managé, Ollama ou Claude.
- **Enterprise** : Kubernetes (API stateless scalable horizontalement, NATS JetStream pour le bus, Qdrant/Kuzu comme services), autoscaling sur le débit d'ingestion.

Roadmap

Phase	Contenu
MVP <i>(livré)</i>	événements seedés, Postgres seul, pipeline in-process, émergence de skills ET d'agents , orchestration bornée, MCP + dashboard SSE, multi-provider (Ollama/Claude/heuristique)
v1 <i>(1-2 mois)</i>	connecteur Slack réel, bus → NATS JetStream, Qdrant, détection de contradictions, résumés multi-niveaux, clustering d'agents par embeddings, vérificateur LLM adversarial, auth + isolation
v2	graphe Kuzu, recherche hybride Tantivy, time-travel/versioning, gouvernance patterns→skills→agents, connecteurs GitHub/Notion/Gmail
Enterprise	RBAC multi-tenant, observabilité complète, K8s, SSO, conformité

Comparaison

Système	Ce qu'il fait	Ce qui manque vs Weave
Notion AI	Q&A sur les docs Notion	pas de mémoire d'événements, pas d'émergence de skills/agents, pas de provenance multi-couche
Glean	recherche d'entreprise fédérée	retrieval de documents, pas de distillation en faits ni d'émergence
Mem0	mémoire pour agents	mémoire <i>par agent</i> , pas de mémoire collective ni d'agents émergents
LangGraph / CrewAI	orchestration d'agents	agents <i>codés à la main</i> ; aucune mémoire organisationnelle partagée sous-jacente

Weave est **orthogonal et complémentaire** : il fournit la *couche mémoire collective* que ces outils supposent mais n'ont pas — et il en fait **émerger** skills et agents plutôt que de les faire coder.

Vision long terme : un OS pour la mémoire organisationnelle

L'étape d'après les agents n'est pas « plus d'agents » : ce sont les **organisations cognitives**. Les agents cessent d'être des assistants isolés pour devenir des **rôles** adossés à une intelligence commune. Chaque interaction enrichit cette intelligence ; les connaissances se consolident ; les habitudes deviennent des patterns ; les procédures récurrentes deviennent des skills ; les domaines récurrents font émerger des agents ; et **chaque nouveau rôle hérite instantanément** de tout l'acquis, sans reprogrammation.

Weave est l'infrastructure de ce futur : un **système d'exploitation pour la mémoire organisationnelle**, où l'objectif n'est pas de conserver des conversations, mais de faire émerger une **compréhension partagée, durable et actionnable** du travail réalisé.

Annexe — Parcours de démonstration (~90 s)

1. Rejouer l'activité : un flux type PennyLane (synchro bancaire Bridge/Budget Insight, Stripe, FEC, runbooks Notion) défile dans le feed live.
2. Les faits/entités/relations se construisent → le knowledge graph grandit.
3. Deux questions récurrentes (« relancer la synchro bancaire », « rejouer un webhook Stripe ») franchissent le seuil → **deux skills émergent**.
4. Le domaine `finance-ops` atteint 2 skills → un **agent spécialiste-finance-ops** émerge (en attente).
5. Approuver l'agent → l'assistant **délègue**, le spécialiste répond, le vérificateur valide → **trace** affichée.
6. Toute la mémoire est aussi interrogeable par un agent externe **via MCP**.